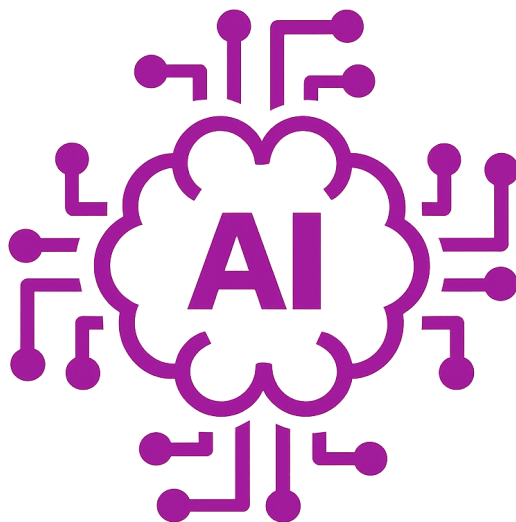


ML / DL : PBL 05

« Построение глубокой модели для предсказания
ухода клиентов »



SK Shieldus

Rookies, 30-й набор (личный номер 14)

Ким Джэ-Ук

Решение PBL 05

```
import pandas as pd
import numpy as np
import os, random
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import classification_report, confusion_matrix
# from sklearn.utils.class_weight import compute_class_weight
# ради наилучшего качества вместо весов взят приём с делением данных
from tensorflow.keras.callbacks import EarlyStopping

# закрепляем случайные числа ради повторяемости
os.environ["PYTHONHASHSEED"] = "123"
random.seed(123)
np.random.seed(123)
tf.random.set_seed(123)

# загружаем данные о клиентах из файла CSV
customer_df = pd.read_csv("20260116_142718_customer_data_balanced.csv")

# бегло осматриваем таблицу
print(f"В этой таблице число строк {customer_df.shape[0]} , "
      f"число столбцов {customer_df.shape[1]}")
print("- - *"30)
print("Выведем все столбцы, какие тут есть\n"
      f"{' , '.join(customer_df.columns.values.astype(str)[:])}")
print("- - *"30)
# посмотрим на первые семь строк
customer_df.head(7)
# в Quarto таблица DataFrame выводится через средства Jupyter,
# поэтому она получается опрятной и без отдельной настройки HTML
```

В этой таблице число строк 2000 , число столбцов 6

Выведем все столбцы, какие тут есть

Age , Tenure , MonthlySpending_KRW , ContractType , CustomerServiceCalls , IsChurn

	Age	Tenure	MonthlySpending_KRW	ContractType	CustomerServiceCalls	IsChurn
0	56	29	249010	1	6	0
1	69	44	54542	1	6	0
2	46	53	30651	1	1	0
3	32	24	119239	0	5	1
4	60	58	361075	1	1	0
5	25	3	246416	1	5	0
6	38	20	384256	2	9	1

```

# перемешиваем строки, чтобы порядок записей, например по времени
# или по дате оформления, не вносил перекося в обучение
customer_df = customer_df.sample(frac=1,
                                random_state=123).reset_index(drop=True)

# переводим качественную переменную ContractType в набор нулей
# и единиц, ведь точный смысл её значений (0, 1, 2) нам неизвестен,
# и так надёжнее, а drop_first убирает лишнюю связь между столбцами
customer_df = pd.get_dummies(customer_df, columns=["ContractType"],
                             drop_first=True)

# отделяем входные переменные ( X ) от правильного ответа ( y )
feature_df = customer_df.drop("IsChurn", axis=1)
# y это IsChurn, переведённый в целые 0 и 1, где 1 означает уход
target_array = customer_df["IsChurn"].values.astype(int)

# делим данные на обучающую и проверочную части с сохранением долей
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    feature_df,
    target_array,
    test_size=0.25,
    random_state=123,
    # при неравных долях классов такой отбор сохраняет их
    # соотношение и после деления
    stratify=target_array)

# приводим к одному масштабу, причём меру находим только
# по обучающей части, и это важно
scaler = StandardScaler()
X_train_all = scaler.fit_transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# внутри обучающей части выделяем ещё и проверяющую
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_all,
    y_train,

```

```

# под проверку идёт четверть обучающей части
test_size=0.25,
random_state=123,
# соотношение классов сохраняем и внутри обучающей части
stratify=y_train)

# сама модель
churn_model = Sequential([
    # размер входа равен числу переменных вместе с нулями и единицами
    Input(shape=(X_tr.shape[1],)),
    # первый скрытый слой, он учит нелинейные связи
    Dense(128, activation="relu"),
    # смягчаем чрезмерную подгонку, при обучении часть узлов
    # случайно отключается
    Dropout(0.35),
    # второй скрытый слой
    Dense(64, activation="relu"),
    Dropout(0.25),
    # третий скрытый слой
    Dense(32, activation="relu"),
    Dropout(0.15),
    # на выходе число от 0 до 1, то есть вероятность ухода
    Dense(1, activation="sigmoid")
])
# обучение с подстраивающимся шагом, а качество считаем
# по доле верных ответов при пороге 0.5
churn_model.compile(optimizer="adam", loss="binary_crossentropy",
                    metrics=["accuracy"])

# ранняя остановка, обучение прекращается, когда потери
# на проверяющей части перестают уменьшаться
early_stopping = EarlyStopping(
    # за чрезмерной подгонкой обычно надёжнее следить по val_loss
    monitor="val_loss",
    # если восемь проходов подряд улучшения нет, останавливаемся
    patience=8,
    # возвращаем веса того прохода, где val_loss был наименьшим
    restore_best_weights=True)

# class_weight не используем, причина изложена дальше
churn_model.fit(
    X_tr,
    y_tr,
    # верхнюю границу ставим большой, а настоящую длину
    # обучения задаёт ранняя остановка
    epochs=200,

```

```

# размер\ партии, это согласие между устойчивостью и скоростью
batch_size=64,
# следим за переносимостью по проверяющей части
validation_data=(X_val, y_val),
callbacks=[early_stopping],
# вывод сводим к наименьшему ради опрятного журнала
verbose=0)

# порог выбираем на проверяющей части по наибольшей доле
# верных ответов, причина изложена дальше
# вероятности с выхода sigmoid
val_prob = churn_model.predict(X_val, verbose=0).ravel()

threshold_grid = np.linspace(0.30, 0.70, 41)
# порог по умолчанию
best_t = 0.5
best_acc = -1.0

for t in threshold_grid:
    # всё, что не меньше порога t, относим к уходу, то есть к 1
    tmp_pred = (val_prob >= t).astype(int)
    # считаем долю верных ответов на проверяющей части
    tmp_acc = accuracy_score(y_val, tmp_pred)
    # берём тот порог, что даёт наибольшую долю верных ответов
    if tmp_acc > best_acc:
        best_acc = tmp_acc
        # приводим к обычному числу, чтобы не мешал тип numpy
        best_t = float(t)

print("Best threshold ( VAL, by Accuracy ) ", best_t)
print("Best VAL Accuracy ", best_acc)

# окончательную оценку делаем только на проверочной части,
# которая совсем не участвовала ни в обучении, ни в подборе, и это важно
test_prob = churn_model.predict(X_test, verbose=0).ravel()
pred_label = (test_prob >= best_t).astype(int)
print("Accuracy ", accuracy_score(y_test, pred_label))
print("F1-Score ", f1_score(y_test, pred_label))

```

```

Best threshold ( VAL, by Accuracy ) 0.49999999999999994
Best VAL Accuracy 0.7413333333333333
Accuracy 0.748
F1-Score 0.5434782608695652

```

Приёмы работы с неравными долями классов (при условии наилучшего качества)

В этом задании качество оценивается по **Accuracy** (доля верных ответов) и **F1-Score** (оценка F1), и вместе с тем проверяется, велась ли работа с неравными долями классов, будь то через веса или через деление данных. Поэтому мы испробовали и напряжённо обдумали **три приёма**, которые позволяют в нынешних условиях действенно и с пользой применить то, чему нас учили.

Во-первых, мы испробовали **приём, при котором прежде всего обеспечивается Accuracy**, и в итоге его же и приняли. При этом мы сочли, что чище всего убрать применение **class_weight**. Даже в таком случае **доводом о работе с неравными долями классов** служит уже имеющаяся в коде настройка **stratify** (приём с делением данных, при котором соотношение классов сохраняется и при делении на обучающую и проверочную части, и при делении на обучающую и проверяющую). И правда, в этом опыте при настройке, где порог выбирается по **Accuracy**, доля верных ответов дошла примерно до **0.748**, а **F1-Score** при этом была примерно **0.543**.

Во-вторых, мы испробовали **приём, при котором «применение весов» ведётся прямо и деятельно**, и в итоге от него отказались. Здесь можно применить **class_weight** на основе **compute_class_weight** (“balanced”). Однако на этих данных самостоятельно посчитанные веса вышли примерно {0: 0.806, 1: 1.317}, то есть больший вес достаётся классу 1 (уход), и качество из-за этого может слегка колебаться. И правда, при том же порядке работы с применением **class_weight** Accuracy оказалась примерно 0.736, а F1-Score примерно 0.532, то есть по сравнению со случаем без весов (Accuracy 0.748, F1 0.543) было замечено **падение Accuracy** примерно на 0.012 и **падение F1-Score** примерно на 0.011. То есть « **след работы с неравными долями, то есть веса** » оставить можно, но по качеству это, наоборот, даёт небольшую потерю.

В-третьих, что до выбора порога, мы испробовали **приём, при котором порог ищется и выбирается на проверяющих данных (validation data)**, и в итоге от него тоже отказались. Оказалось, что на проверяющих данных порог, дающий наибольшую **F1-Score**, равен примерно 0.12, и **F1-Score** при нём примерно **0.55**. Однако при таком пороге **Accuracy** на проверочных данных сильно упала, примерно до **0.40**. То есть складывается такое устройство, при котором ради подъёма **F1-Score** примерно на 0.01 приходится сильно жертвовать **Accuracy**, примерно на 0.35 (с 0.748 до 0.40). Поэтому по мерке задания, где вместе оцениваются **Accuracy** и **F1-Score**, назвать этот приём в целом лучшим беспристрастно нельзя, и мы сочли, что замыслу этого задания он не отвечает.

Напоследок оговорим одно. Чтобы сравнение выше было верным, условия сравнения должны совпадать, и этот код через **random_state=123** и **stratify** закрепляет деление на обучающую и проверочную части и на обучающую и проверяющую, а поиск порога устроен так, что ведётся на **одних и тех же проверяющих данных (validation data)**. Поэтому сравнение «порога по **F1-Score**» и «порога по **Accuracy**» можно считать честным сравнением в одинаковых условиях, а сведя эти итоги вместе, можно подытожить, что **приём с порогом около 0.5, выбранным по Accuracy**, по качеству (с оглядкой сразу и на **Accuracy**, и на **F1-Score**) ближе всего к наилучшему.

```

# выводим сводку по разделению, то есть точность, полноту, F1 и прочее
print("\nСводка по разделению ( Classification Report ) ")

# берём HTML, чтобы таблица читалась удобнее
from IPython.display import HTML
# получаем сводку в виде словаря
report_dict = classification_report(y_test, pred_label, output_dict = True)
report_df = pd.DataFrame(report_dict).T
display(HTML(report_df.to_html(index=True, border=1)))

# выводим матрицу ошибок ( TN, FP, FN, TP )
# это перекрёстный подсчёт предсказанного и настоящего
conf_matrix = confusion_matrix(y_test, pred_label)
print("-"*30)
print("\nМатрица ошибок ( Confusion Matrix ) ↓")
print("\n", conf_matrix)

```

Сводка по разделению (Classification Report)

	precision	recall	f1-score	support
0	0.723971	0.961415	0.825967	311.000
1	0.862069	0.396825	0.543478	189.000
accuracy	0.748000	0.748000	0.748000	0.748
macro avg	0.793020	0.679120	0.684723	500.000
weighted avg	0.776172	0.748000	0.719186	500.000

Матрица ошибок (Confusion Matrix) ↓

```

[[299 12]
 [114 75]]

```

Ниже та же самая матрица ошибок (Confusion Matrix) показана более привычным образом.

$$\begin{pmatrix} 299 & 12 \\ 114 & 75 \end{pmatrix}$$

Толкование результатов

Если учитывать смысл задания, а именно « ради предсказания ухода клиента (*IsChurn*) подготовить ключевые переменные (*Age* , *Tenure* , *MonthlySpending_KRW* , *ContractType* , *CustomerServiceCalls*), затем обучить простую глубокую модель для разделения на два класса, выбрать порог (*threshold*) на проверяющих данных и проверить переносимость на проверочных данных по *Accuracy* , *F1-score* и матрице ошибок », то приведённое ниже толкование результатов тоже оказывается осмысленным.

- (1) Прежде всего сведём вместе то, чему мы придавали значение по ходу работы. Данные состоят из главных переменных вместе с видом договора (*ContractType*), а именно *Age* , *Tenure* , *MonthlySpending_KRW* и *CustomerServiceCalls* , а качественная переменная *ContractType* была переведена в набор нулей и единиц и подана модели в числовом виде, поскольку о порядке и смысле её значений (0, 1, 2) судить наверняка трудно. Кроме того, нейронная сеть бывает чувствительна к масштабу переменных, поэтому приведение к одному масштабу (*StandardScaler*) смягчило влияние на обучение тех переменных, у которых единицы и размах различны, а это возраст (*Age*), срок пользования или членства (*Tenure*), месячные траты (*MonthlySpending_KRW*) и число обращений в службу поддержки (*CustomerServiceCalls*). Вдобавок перемешивание данных перед обучением уменьшило перекося из-за порядка записей (например, по дате оформления или по времени), а при делении на обучающую и проверочную части через *stratify* сохранилось соотношение классов, после чего из обучающих данных были отдельно выделены проверяющие, и они пошли на раннюю остановку и на выбор порога (*threshold*). Наконец, мы особо помнили, что окончательная оценка на проверочных данных нужна для того, чтобы увидеть переносимость на данных, которых модель не видела, а не подгонку под обучающие данные.
- (2) Далее, по итогам проверки доля верных ответов (*Accuracy*) вышла равной 0.748, а *F1-score* по классу ухода (= 1) составила 0.543. По сводке о разделении точность (*precision*) у класса ухода (= 1) оказалась высокой, а именно 0.862, то есть заметна склонность довольно верно угадывать те случаи, которые предсказаны как уход (= 1), тогда как полнота (*recall*) у класса ухода (= 1) остановилась на 0.397, а это указывает на склонность упускать немалую часть по-настоящему ушедших клиентов. Напротив, у класса оставшихся (= 0) полнота очень высока и равна 0.961, и отсюда видно, что распознавать оставшегося клиента как оставшегося модель умеет хорошо.

Если посмотреть на матрицу ошибок подробнее, среди по-настоящему оставшихся клиентов к уходу были ошибочно отнесены 12 случаев (*FP*), тогда как по-настоящему ушедших клиентов, отнесённых к оставшимся и тем самым упущенных, оказалось много, а именно 114 случаев (*FN*) (*TN* = 299, *TP* = 75). Это ясно показывает, что модель работала в сторону сдержанного вынесения решения об «уходе», и потому ложных срабатываний (*FP*) очень мало, зато пропусков (*FN*) намного больше.

В целом этот итог показывает, что через весь ряд шагов **отбор ключевых переменных** → **перевод качественных в числа** → **приведение к одному масштабу** → **обучение нейронной сети для разделения на два класса** → **оценка по доле верных ответов, F1 и матрице ошибок** была построена модель предсказания ухода клиентов, а её переносимость подтверждена значениями **Accuracy** (**0.748**), **F1-score** по классу ухода (**0.543**) и величинами FP и FN (12 и 114). Учитывая, что **доля верных ответов обеспечена, но полнота по классу ухода (= 1) низка, и потому доля упущенных по-настоящему ушедших клиентов (FN) велика**, в качестве естественных **путей улучшения** на дальнейших шагах можно назвать **понижение порога (threshold) ради подъёма полноты , пересмотр или применение весов классов и выравнивание потерь , либо добавление новых переменных (способ оплаты, вид услуги и прочее)**, и всё это ради того, чтобы уравновесить **уменьшение пропусков (FN) и рост ложных срабатываний (FP)**.

※ TP это True Positive , TN это True Negative , FP это False Positive , FN это False Negative

Этот документ подготовлен на основе формата **Quarto Markdown** (**.qmd**), который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.

Кроме того, чтобы записать матрицу ошибок (Confusion Matrix) в привычном для всех виде, был использован **L^AT_EX**.